

How to implement a new PDE

Manfred Kaltenbacher and Andreas Hüppe

Institute of Mechanics and Mechatronics, Vienna University of Technology

Austria

□ Let's define a simple parabolic PDE

$$\alpha \frac{\partial u}{\partial t} - \nabla \cdot \beta \nabla u = f; \quad u = u_e \text{ on } \Gamma$$

□ Weak form

$$\int_{\Omega} \alpha v \frac{\partial u}{\partial t} d\mathbf{x} + \int_{\Omega} \beta \nabla v \cdot \nabla u d\mathbf{x} = \int_{\Omega} v f d\mathbf{x}$$

$$u = u_e \text{ on } \Gamma$$

Main Steps

❑ Directory source/PDE

- Create the Files „TestPDE.cc“ and „TestPDE.hh“ (perhaps it makes sense to copy a similar already existing PDE).
- Include „TestPDE.cc“ in the file „CMakeLists.txt“ so that cmake is also compiling this file.

❑ Subversion

- Add the files to the repository (performed at the next check in)
svn add TestPDE.cc TestPDE.hh

❑ Start „eclipse“ and open the files „TestPDE.hh“ and „TestPDE.cc“

Main Steps

❑ Important aspects for „TestPDE.hh“

- Put a short documentation

```
/*!  
 *   \file   TestPDE.hh  
 *   \brief  This PDE will be used to train new developers  
 *  
 *   \date   July 2, 2013  
 *   \author  Manfred Kaltenbacher, TU Wien  
 */
```

- As much as possible use forward class declaration instead of including *.hh files
- Define the hh-File and use our namespace

```
#ifndef TESTPDE_HH_  
#define TESTPDE_HH_  
  
namespace CoupledField {  
...  
...  
}
```

Main Steps

❑ Important aspects for „TestPDE.hh“

➤ Main Methods

```
public:
    //! Constructor.
    TestPDE( Grid* aGrid, PtrParamNode paramNode, PtrParamNode infoNode,
            shared_ptr<SimState> simState, Domain* domain );

    //! Destructor
    virtual ~TestPDE(){};

protected:
    //! Initialize NonLinearities
    virtual void InitNonLin();

    //! \copydoc SinglePDE::CreateFeSpaces
    virtual std::map<SolutionType, shared_ptr<FeSpace> >
    CreateFeSpaces( const std::string& formulation, PtrParamNode infoNode );

    //! define all (bilinearform) integrators needed for this pde
    void DefineIntegrators();

    //! define surface integrators needed for this pde
    void DefineSurfaceIntegrators( ) {};
```

← We have no surface operator to be implemented.

Main Steps

❏ Important aspects for „TestPDE.hh“

➤ Main methods (cont.)

```
//! Defines the integrators needed for ncInterfaces  
void DefineNcIntegrators() {};
```

```
//! Define all RHS linearforms for load / excitation  
void DefineRhsLoadIntegrators();
```

```
//! define the SoltionStep-Driver  
void DefineSolveStep();
```

```
//! Define available postprocessing results  
void DefinePrimaryResults();
```

```
//! Define available postprocessing results  
void DefinePostProcResults();
```

```
//! Init the time stepping  
void InitTimeStepping();
```

Main Steps

❑ Important aspects for „TestPDE.hh“

➤ Constructor

```
//! Constructor
/*!
\param grid pointer to grid
\param paramNode pointer to the corresponding parameter node
\param infoNode pointer for writing to info (*.info.xml) file
\param simState pointer to state for saving / restoring
\param domain pointer to domain object
*/
TestPDE( Grid* aGrid, PtrParamNode paramNode,
        PtrParamNode infoNode,
        shared_ptr<SimState> simState, Domain* domain );
```

- **Grid* aGrid:** pointer to the computational grid
- **PtrParamNode paramNode:** pointer to xml for getting xml-definitions
- **PtrParamNode infoNode:** for writing to *.info.xml file
- **shared_ptr<SimState> simState:** pointer to the state; has the functionality for saving and restoring the internal state of a simulation at any given time / frequency step.
- **Domain* domain:** pointer to the domain

Main Steps

❑ Be aware: a lot of actings happen in

- SinglePDE, StdPDE and BasePDE (have a look to all these classes!)
- E.g., constructor of StdPDE

```
StdPDE::StdPDE(Grid *aptgrid, PtrParamNode paramNode, PtrParamNode infoNode,  
               shared_ptr<SimState> simState, Domain* domain ) :  
    BasePDE(paramNode, infoNode, simState, domain),  
    ptGrid_(aptgrid),  
    subType_(),  
    nonLin_(false),  
    nonLinMaterial_(false),  
    isHysteresis_(false),  
    isIterCoupled_(false),  
    diagMass_(false),  
    needsAlgsys_(true),  
    isAlwaysStatic_(false),  
    dim_(ptGrid_->GetDim()),  
    isaxi_(ptGrid_->IsAxi()),  
    isComplex_(false),  
    assemble_(NULL),  
    solveStep_(NULL),  
    algsys_(NULL)  
{  
}
```


Main Steps

❑ Constructor of TestPDE

```
TestPDE::TestPDE(Grid * aptgrid, PtrParamNode paramNode,  
                PtrParamNode infoNode,  
                shared_ptr<SimState> simState, Domain* domain)  
:SinglePDE( aptgrid, paramNode, infoNode, simState, domain ) {  
  
    pdename_      = "testPDE";  
    pdematerialclass_ = TESTMAT;  
    nonLin_       = false;  
  
    //! Always use updated Lagrangian formulation  
    updatedGeo_   = true;  
}
```

Efforts action concerning the material class

1. In „Environment.hh“: extend enumeration „MaterialClass“ by *TESTMAT*
 - String name: testmat (used in xml-material file)
2. In „Environment.cc“: fill out the corresponding „String2Enum“ and „Enum2String“
3. In „Environment.hh“: define the material parameters alpha and beta in „MaterialType“
4. In „Environment.cc“: fill out the „MaterialTypeEnum.Add“
 - Name: TEST_ALPHA, TEST_BETA

Main Steps

❑ Define the functional space

➤ Method CreateFeSpaces

```
std::map<SolutionType, shared_ptr<FeSpace> >
TestPDE::CreateFeSpaces(const std::string& formulation, PtrParamNode infoNode) {
    std::map<SolutionType, shared_ptr<FeSpace> > crSpaces;
    if(formulation == "default" || formulation == "H1"){
        PtrParamNode potSpaceNode = infoNode->Get("testDof");
        crSpaces[TEST_DOF] =
            FeSpace::CreateInstance(myParam_,potSpaceNode,FeSpace::H1, ptGrid_);
        crSpaces[TEST_DOF]->Init(solStrat_);
    }else{
        EXCEPTION("The formulation " << formulation << "of TEST PDE is not known!");
    }
    return crSpaces;
}
```

➤ Needs definition of primary degree of freedom

- In „Environment.hh“: define „SolutionType“ (primary dof and, e.g., postprocessing result, and any right hand side load)
- We name them „TEST_DOF“ and „TEST_FIELD“ and „TEST_RHS_LOAD“
- In „Environment.cc“: put them to the „SolutionTypeEnum“

Main Steps

❑ Define result types

➤ Method DefinePrimaryResults

```
void TestPDE::DefinePrimaryResults() {  
  
    // === TEST ===  
    shared_ptr<ResultInfo> res1( new ResultInfo);  
    res1->resultType = TEST_DOF;  
  
    res1->dofNames = "";  
    res1->unit = "?";  
    res1->definedOn = ResultInfo::NODE;  
    res1->entryType = ResultInfo::SCALAR;  
    feFunctions_[TEST_DOF]->SetResultInfo(res1);  
    results_.Push_back( res1 );  
    availResults_.insert( res1 );  
    res1->SetFeFunction(feFunctions_[TEST_DOF]);  
    DefineFieldResult( feFunctions_[TEST_DOF], res1 );  
  
    // -----  
    // Define xml-names of Dirichlet BCs  
    // -----  
    hdbcSolNameMap_[TEST_DOF] = "testGround";  
    idbcSolNameMap_[TEST_DOF] = "testPotential";
```

Tag in xml-File

Main Steps

❑ Define result types

➤ Method DefinePostProcResults

```
void TestPDE::DefinePostProcResults() {  
    // === TEST FIELD ===  
    StdVector<std::string> vecComponents;  
    if( dim_ == 3 )  
        vecComponents = "x", "y", "z";  
    else if ( isaxi_ )  
        vecComponents = "r", "z";  
    else  
        vecComponents = "x", "y";  
  
    shared_ptr<BaseFeFunction> feFct = feFunctions_[TEST_DOF];  
    shared_ptr<ResultInfo> field(new ResultInfo);  
    field->resultType = TEST_FIELD;  
    field->dofNames = vecComponents;  
    field->unit = "??";  
    field->definedOn = ResultInfo::ELEMENT;  
    field->entryType = ResultInfo::VECTOR;  
    availResults_.insert( field );  
}
```

Main Steps

❑ Define result types

➤ Method DefinePostProcResults (cont.)

```
shared_ptr<CoefFunctionFormBased> fieldFunc;  
if( isComplex_ )  
    fieldFunc.reset(new CoefFunctionBOP<Complex>(feFct, field));  
else  
    fieldFunc.reset(new CoefFunctionBOP<Double>(feFct, field));  
DefineFieldResult( fieldFunc, field );  
stiffFormCoefs_.insert(fieldFunc);  
  
}
```

Main Steps

❑ Define the bilinear forms: Method DefineIntegrators()

➤ Define local variables

```
RegionIdType actRegion;  
BaseMaterial * actSDMat = NULL;  
  
// Define integrators for all (material) regions  
std::map<RegionIdType, BaseMaterial*>::iterator it;  
  
//get FEFunctor and space  
shared_ptr<BaseFeFunction> feFunc = feFunctions_[TEST_DOF];  
shared_ptr<FeSpace> mySpace = feFunc->GetFeSpace();
```

➤ Loop over all regions, the PDE is defined on

```
for ( it = materials_.begin(); it != materials_.end(); it++ ) {  
    // Set current region and material  
    actRegion = it->first;  
    actSDMat = it->second;  
  
    // Get current region name  
    std::string regionName = ptGrid_->GetRegion().ToString(actRegion);  
  
    // create new entity list  
    shared_ptr<ElemList> actSDList( new ElemList(ptGrid_ ) );  
    actSDList->SetRegion( actRegion );
```

Main Steps

❑ Define the bilinear forms: Method DefineIntegrators() (cont.)

➤ Set the FE ansatz

```
// --- Set the FE ansatz for the current region ---
PtrParamNode curRegNode = myParam_->Get("regionList")->
    GetByVal("region","name",regionName.c_str());
std::string polyId = curRegNode->Get("polyId")->As<std::string>();
std::string integId = curRegNode->Get("integId")->As<std::string>();
mySpace->SetRegionApproximation(actRegion, polyId,integId);

// pass entitylist of fespace / fefunction
feFunc->AddEntityList( actSDList );
```

➤ Define and set the bilinear form of the stiffness matrix

```
PtrCoefFct beta = actSDMat->GetScalCoefFnc( TEST_BETA, Global::REAL );

BaseBDBInt* stiffInt = NULL;
if( dim_ == 2 ) {
    stiffInt = new BBInt<>(new GradientOperator<FeH1,2>(), beta,1.0, updatedGeo_ );
} else {
    stiffInt = new BBInt<>(new GradientOperator<FeH1,3>(), beta,1.0, updatedGeo_ );
}
stiffInt->SetName("StiffnessIntegrator");
```

Main Steps

❑ Define the bilinear forms: Method DefineIntegrators() (cont.)

➤ Define and set the bilinear form of the stiffness matrix (cont.)

```
BiLinFormContext * stiffIntDescr = new BiLinFormContext(stiffInt, STIFFNESS );  
  
//stiffIntDescr->SetPtPdes(this, this);  
stiffIntDescr->SetEntities( actSDList, actSDList );  
stiffIntDescr->SetFeFunctions(feFunc,feFunc);  
stiffInt->SetFeSpace( mySpace );  
  
assemble_->AddBiLinearForm( stiffIntDescr );  
bdbInts_[actRegion] = stiffInt;
```

For postprocessing

➤ Just the same do for the bilinear form of the mass matrix (material parameter alpha)

Main Steps

- ❑ Extend the material class: directory „source/Materials“
 - Generate the files „testMaterial.hh“ and „TestMaterial.cc“
 - Add the file „Testmaterial.cc“ to the CMakeLists.txt
 - Class TestMaterial

```
class TestMaterial : public BaseMaterial {
public:

    //! Default constructor
    TestMaterial(MathParser* mp, CoordSystem * defaultCoosy);

    //! Destructor
    ~TestMaterial();

    //! set a scalar real material parameter
    void SetScalar( Double param, MaterialType matType, Global::ComplexPart dataType );

    //! get a scalar real material parameter
    void GetScalar( Double& param, MaterialType matType, Global::ComplexPart dataType )
    const;

private:
};
```

Main Steps

❑ Extend the material class: directory „source/Materials“ (cont.)

- Svn add TestMaterial.cc TestMaterial.hh
- Class TestMaterial: Construtor

```
TestMaterial::TestMaterial(MathParser* mp, CoordSystem * defaultCoosy)
    : BaseMaterial(mp, defaultCoosy) {

    materialDatabaseName_ = "Test";

    //set the allowed material parameters
    isAllowed_.insert( TEST_ALPHA );
    isAllowed_.insert( TEST_BETA );
}
```

❑ Extend class „XMLMaterialHandler“ in source/DataInOut/ParamHandling/

- Add in „XMLMaterialHandler.cc“ #include "Materials/TestMaterial.hh"

```
➤ else if ( matClass == TEST ) {
    material = new TestMaterial(mp, cs);
    ReadTest( material, pn );
}
```

↗
Add declaration to XMLMaterialHandler.hh

Main Steps

❑ Class „XMLMaterialHandler“ in source/DataInOut/ParamHandling/ (cont.)

➤ Method ReadTest

```
void XMLMaterialHandler::ReadTest(BaseMaterial *material, PtrParamNode test)
{
    // read alpha
    if(test->Has("alpha")) {
        PtrCoefFct alphaFct =
            CoefFunction::Generate(mp_, Global::REAL,
                                   test->Get("alpha")->As<std::string>() );
        material->SetCoefFct( TEST_ALPHA, alphaFct );
    }

    // read beta
    if(test->Has("beta")) {
        PtrCoefFct betaFnc =
            CoefFunction::Generate(mp_, Global::REAL,
                                   test->Get("beta")->As<std::string>() );
        material->SetCoefFct( TEST_BETA, betaFnc );
    }
}
```

Main Steps

❑ Extend the XML-Scheme: see share/xml/CFS-Simulation/Schemas/

- Generate the file „CFS_PDEtest.xsd“
- Main entries

```
<xsd:complexType name="DT_PDETest">
  <xsd:complexContent>
    <xsd:extension base="DT_PDEBasic">
      <xsd:sequence>

        <!-- Regions the PDE lives on -->
        <xsd:element name="regionList" minOccurs="1" maxOccurs="1">
          <xsd:complexType>
            <xsd:sequence>
              <xsd:element name="region" minOccurs="1" maxOccurs="unbounded">
                <xsd:complexType>
                  <xsd:attribute name="name" type="xsd:token" use="required"/>
                  <xsd:attribute name="polyId" type="xsd:string" use="optional"
                                default="default"> </xsd:attribute>
                  <xsd:attribute name="integId" type="xsd:string" use="optional"
                                default="default"> </xsd:attribute>
                </xsd:complexType>
              </xsd:element>
            </xsd:sequence>
          </xsd:complexType>
        </xsd:element>
      </xsd:sequence>
    </xsd:complexType>
  </xsd:element>
```

Main Steps

❑ Extend the XML-Scheme: see share/xml/CFS-Simulation/Schemas/

➤ Main entries (cont.)

```
<!-- Boundary Conditions & Loads (optional) -->
<xsd:element name="bcsAndLoads" minOccurs="0" maxOccurs="unbounded">
  <xsd:complexType>
    <xsd:choice minOccurs="0" maxOccurs="unbounded">
      <!-- Dirichlet Boundary Conditions -->
      <xsd:element name="testGround" type="DT_BcHomScalar"/>
      <xsd:element name="testPotential" type="DT_BcInhomScalar"/>
      <!-- test specific sources -->
      <xsd:element name="testSourceDensity" type="DT_BcInhomScalar"/>
    </xsd:choice>
  </xsd:complexType>
</xsd:element>
```

➤ Svn add CFS_PDEtest.xsd

➤ Add in CFS.xsd

- <xsd:include schemaLocation="Schemas/CFS_PDEtest.xsd"/>

➤ Add in CMakeLists.txt (in share/xml/)

- COMMAND \${CMAKE_COMMAND} -E copy_if_different
\${SRC_DIR}/Schemas/CFS_PDEtest.xsd \${DST_DIR}/Schemas/CFS_PDEtest.xsd

Main Steps

❑ Extend the XML-Scheme: see share/xml/CFS-Material/Schemas/

- Generate the file „CFS_MATtest.xsd“
- Main entries

```
<xsd:complexType name="DT_MatTest">  
  <xsd:sequence>  
    <xsd:element name="alpha" type="xsd:token">  
    </xsd:element>  
  
    <xsd:element name="beta" minOccurs="0" type="xsd:token">  
    </xsd:element>  
  </xsd:sequence>  
</xsd:complexType>
```

- svn add CFS_MATtest.xsd
- Add in „CFS_Material.xsd“ (in share/xml/CFS-Material/)
 - <xsd:include schemaLocation="Schemas/CFS_MATtest.xsd"/>
 - <xsd:element name="test" type="DT_MatTest" minOccurs="0" maxOccurs="1" />
- Add to CMakeLists.txt (in share/xml/)
 - COMMAND \${CMAKE_COMMAND} -E copy_if_different
 \${SRC_DIR}/Schemas/CFS_MATtest.xsd \${DST_DIR}/Schemas/CFS_MATtest.xsd

Calling Structure

- ❑ In directory “source/Domain” add in “Domain.cc”

- #include "PDE/TestPDE.hh"
- In method CreateSinglePDEs

```
else if (actPdeName == "testPDE") {  
    ptSinglePde_[i] = new TestPDE(defaultGrid, actPdeNode, infoNode,  
        simState_, this );  
}
```

- ❑ Time stepping

- Define SolveStep

```
void TestPDE::DefineSolveStep() {  
    solveStep_ = new StdSolveStep(*this);  
}
```

- Init

```
void TestPDE::InitTimeStepping() {  
    shared_ptr<BaseTimeScheme>  
        myScheme(new TimeSchemeGLM(GLMScheme::TRAPEZOIDAL, 0) );  
    feFunctions_[TEST_DOF]->SetTimeScheme(myScheme);  
}
```

- ❑ Add RHS-linearform (): file TestPDE.cc
 - Method DefineRhsLoadIntegrators (just main part)

```
LinearForm * lin = NULL;
ReadRhsExcitation( "testSourceDensity", dofNames,
                  ResultInfo::VECTOR, isComplex_, ent, coef, coefUpdateGeo );
for( UInt i = 0; i < ent.GetSize(); ++i ) {
    if (ent[i]->GetType() == EntityList::NODE_LIST)
        EXCEPTION("Test source density must be defined on elements")

    EntityIterator it = ent[i]->GetIterator();
    it.Begin();
    if ( isComplex_ )
        lin = new BUIntegrator<IdentityOperator<FeH1>, Complex>(Complex{1.0}, coef[i], coefUpdateGeo);
    else
        lin = new BUIntegrator<IdentityOperator<FeH1>, Double>(1.0, coef[i], coefUpdateGeo);

    lin->SetName("TestSourceDensityInt");
    LinearFormContext *ctx = new LinearFormContext( lin );
    ctx->SetEntities( ent[i] );
    ctx->SetFeFunction(myFct);
    assemble_->AddLinearForm(ctx);
} // for
```